

Package ‘CohortMethod’

June 19, 2026

Type Package

Title Comparative Cohort Method with Large Scale Propensity and Outcome Models

Version 6.0.3

Date 2026-05-19

Maintainer Martijn Schuemie <schuemie@ohdsi.org>

Description Functions for performing comparative cohort studies in an observational database in the Observational Medical Outcomes Partnership (OMOP) Common

Data Model. Can extract all necessary data from a database. This implements large-scale propensity scores (LSPS) as described in Tian et al. (2018) <[doi:10.1093/ije/dyy120](https://doi.org/10.1093/ije/dyy120)>, using a large set of covariates, including for example all drugs, diagnoses, procedures, as well as age, comorbidity indexes, etc. Large scale regularized regression is used to fit the propensity and outcome models as described in Suchard et al. (2013) <[doi:10.1145/2414416.2414791](https://doi.org/10.1145/2414416.2414791)>.

Functions are included for trimming, stratifying, (variable and fixed ratio) matching and weighting by propensity scores, as well as diagnostic functions, such as propensity score distribution

plots and plots showing covariate balance before and after matching and/or trimming. Supported outcome models are (conditional) logistic regression, (conditional) Poisson regression, and (stratified) Cox regression. Also included are Kaplan-Meier plots that can adjust for the stratification or matching.

License Apache License 2.0

VignetteBuilder knitr

URL <https://ohdsi.github.io/CohortMethod/>, <https://github.com/OHDSI/CohortMethod>

BugReports <https://github.com/OHDSI/CohortMethod/issues>

Depends R (>= 4.1.0),
DatabaseConnector (>= 6.0.0),
Cyclops (>= 3.6.0),
FeatureExtraction (>= 3.0.0),
Andromeda (>= 0.6.3)

Imports methods,
utils,
ggplot2,
gridExtra,
grid,
readr,

plyr,
 dplyr,
 rlang,
 Rcpp ($\geq 0.11.2$),
 SqlRender ($\geq 1.18.0$),
 survival,
 ParallelLogger ($\geq 3.4.2$),
 checkmate,
 EmpiricalCalibration,
 jsonlite,
 R6,
 digest

Suggests testthat,
 pROC,
 knitr,
 rmarkdown,
 Eunomia,
 zip,
 withr,
 R.utils,
 RSQLite,
 ResultModelManager,
 markdown,
 PSweight

SystemRequirements Java

LinkingTo Rcpp

NeedsCompilation yes

Roxygen list(markdown = TRUE)

Encoding UTF-8

Config/roxygen2/version 8.0.0

Contents

adjustedKm	4
checkCmInstallation	4
CohortMethodData-class	5
cohortMethodDataSimulationProfile	5
computeCovariateBalance	5
computeEquipoise	8
computeMdr	8
computePsAuc	9
convertUntypedListToCmAnalysesSpecifications	10
createCmAnalysesSpecifications	11
createCmAnalysis	12
createCmDiagnosticThresholds	13
createCmTable1	14
createCohortMethodDataSimulationProfile	15
createComputeCovariateBalanceArgs	16
createCreatePsArgs	17
createCreateStudyPopulationArgs	18

createDefaultMultiThreadingSettings	19
createFitOutcomeModelArgs	20
createGetDbCohortMethodDataArgs	21
createMatchOnPsArgs	23
createMultiThreadingSettings	24
createOutcome	25
createPs	26
createResultsDataModel	27
createStratifyByPsArgs	28
createStudyPopulation	28
createTargetComparatorOutcomes	29
createTrimByPsArgs	30
createTruncateIptwArgs	31
drawAttritionDiagram	32
exportToCsv	33
fitOutcomeModel	33
getAttritionTable	34
getDataMigrator	35
getDbCohortMethodData	35
getDefaultCmTable1Specifications	37
getDiagnosticsSummary	37
getFileReference	38
getFollowUpDistribution	38
getGeneralizabilityTable	39
getInteractionResultsSummary	40
getOutcomeModel	40
getPsModel	41
getResultsDataModelSpecifications	41
getResultsSummary	42
isCohortMethodData	42
loadCmAnalysisList	43
loadCohortMethodData	43
loadTargetComparatorOutcomesList	44
matchOnPs	44
migrateDataModel	45
plotCovariateBalanceOfTopVariables	46
plotCovariateBalanceScatterPlot	47
plotCovariatePrevalence	48
plotFollowUpDistribution	49
plotKaplanMeier	50
plotPs	51
plotTimeToEvent	53
runCmAnalyses	54
saveCmAnalysisList	56
saveCohortMethodData	56
saveTargetComparatorOutcomesList	57
simulateCohortMethodData	57
stratifyByPs	58
trimByPs	59
truncateIptw	60
uploadResults	60

adjustedKm	<i>Compute a weight-adjusted Kaplan-Meier curve</i>
------------	---

Description

Compute a weight-adjusted Kaplan-Meier curve

Usage

```
adjustedKm(weight, time, y)
```

Arguments

weight	Vector of observation weights
time	Vector of event times
y	Vector outcomes (0 indicates censoring, 1 indicates event-of-interest)

checkCmInstallation	<i>Check is CohortMethod and its dependencies are correctly installed</i>
---------------------	---

Description

Check is CohortMethod and its dependencies are correctly installed

Usage

```
checkCmInstallation(connectionDetails)
```

Arguments

connectionDetails	An R object of type connectionDetails created using the function createConnectionDetails in the DatabaseConnector package.
-------------------	--

Details

This function checks whether CohortMethod and its dependencies are correctly installed. This will check the database connectivity, large scale regression engine (Cyclops), and large data object handling (ff).

CohortMethodData-class

Cohort Method Data

Description

CohortMethodData is an S4 class that inherits from [CovariateData](#), which in turn inherits from [Andromeda](#). It contains information on the cohorts, their outcomes, and baseline covariates. Information about multiple outcomes can be captured at once for efficiency reasons.

A CohortMethodData is typically created using [getDbCohortMethodData\(\)](#), can only be saved using [saveCohortMethodData\(\)](#), and loaded using [loadCohortMethodData\(\)](#).

Usage

```
## S4 method for signature 'CohortMethodData'
show(object)
```

```
## S4 method for signature 'CohortMethodData'
summary(object)
```

Arguments

object An object of type CohortMethodData.

cohortMethodDataSimulationProfile

A simulation profile

Description

A simulation profile

Usage

```
data(cohortMethodDataSimulationProfile)
```

computeCovariateBalance

Compute covariate balance before and after PS adjustment

Description

For every covariate, prevalence in treatment and comparator groups before and after matching/trimming/weighting are computed. When variable ratio matching was used the balance score will be corrected according to the method described in Austin et al (2008).

Usage

```
computeCovariateBalance(
  population,
  cohortMethodData,
  computeCovariateBalanceArgs = createComputeCovariateBalanceArgs()
)
```

Arguments

population A data frame containing the people that are remaining after PS adjustment.

cohortMethodData An object of type [CohortMethodData](#) as generated using [getDbCohortMethodData\(\)](#).

computeCovariateBalanceArgs Settings object as created by [createComputeCovariateBalanceArgs\(\)](#).

Details

The population data frame should have the following three columns:

- **rowId** (numeric): A unique identifier for each row (e.g. the person ID).
- **treatment** (integer): Column indicating whether the person is in the target (1) or comparator (0) group.
- **propensityScore** (numeric): Propensity score.

Value

Returns a tibble describing the covariate balance before and after PS adjustment, with one row per covariate, with the same data as the `covariateRef` table in the `CohortMethodData` object, and the following additional columns:

- **beforeMatchingMeanTarget**: The (weighted) mean value in the target before PS adjustment.
- **beforeMatchingMeanComparator**: The (weighted) mean value in the comparator before PS adjustment.
- **beforeMatchingSumTarget**: The (weighted) sum value in the target before PS adjustment.
- **beforeMatchingSumComparator**: The (weighted) sum value in the comparator before PS adjustment.
- **beforeMatchingSdTarget**: The standard deviation of the value in the target before PS adjustment.
- **beforeMatchingSdComparator**: The standard deviation of the value in the comparator before PS adjustment.
- **beforeMatchingMean**: The mean of the value across target and comparator before PS adjustment.
- **beforeMatchingSd**: The standard deviation of the value across target and comparator before PS adjustment.
- **beforeMatchingStdDiff**: The standardized difference of means when comparing the target to the comparator before PS adjustment.
- **beforeMatchingSdmVariance**: The variance of the standardized difference of the means when comparing the target to the comparator before PS adjustment.
- **beforeMatchingSdmP**: The P-value for whether `abs(beforeMatchingStdDiff)` exceeds the threshold.

- beforeMatchingBalanced : TRUE if the covariate is considered balanced between the target and comparator before PS adjustment (depending on the threshold and alpha settings).
- afterMatchingMeanTarget: The (weighted) mean value in the target after PS adjustment.
- afterMatchingMeanComparator: The (weighted) mean value in the comparator after PS adjustment.
- afterMatchingSumTarget: The (weighted) sum value in the target after PS adjustment.
- afterMatchingSumComparator: The (weighted) sum value in the comparator after PS adjustment.
- afterMatchingSdTarget: The standard deviation of the value in the target after PS adjustment.
- afterMatchingSdComparator: The standard deviation of the value in the comparator after PS adjustment.
- afterMatchingMean: The mean of the value across target and comparator after PS adjustment.
- afterMatchingSd: The standard deviation of the value across target and comparator after PS adjustment.
- afterMatchingStdDiff: The standardized difference of means when comparing the target to the comparator after PS adjustment.
- afterMatchingSdmVariance: The variance of the standardized difference of the means when comparing the target to the comparator after PS adjustment.
- afterMatchingSdmP : The P-value for whether abs(afterMatchingStdDiff) exceeds the threshold.
- afterMatchingBalanced : TRUE if the covariate is considered balanced between the target and comparator before PS adjustment (depending on the threshold and alpha settings).
- targetStdDiff: The standardized difference of means when comparing the target before PS adjustment to the target after PS adjustment.
- comparatorStdDiff: The standardized difference of means when comparing the comparator before PS adjustment to the comparator after PS adjustment. -targetComparatorStdDiff: The standardized difference of means when comparing the entire population before PS adjustment to the entire population after PS adjustment.

The 'beforeMatchingStdDiff' and 'afterMatchingStdDiff' columns inform on the balance: are the target and comparator sufficiently similar in terms of baseline covariates to allow for valid causal estimation?

The 'targetStdDiff', 'comparatorStdDiff', and 'targetComparatorStdDiff' columns inform on the generalizability: are the cohorts after PS adjustment sufficiently similar to the cohorts before adjustment to allow generalizing the findings to the original cohorts?

References

- Austin, PC (2008) Assessing balance in measured baseline covariates when using many-to-one matching on the propensity-score. *Pharmacoepidemiology and Drug Safety*, 17: 1218-1225.
- Hripcsak G, Zhang L, Chen Y, Li K, Suchard MA, Ryan PB, Schuemie MJ (2025) Assessing Covariate Balance with Small Sample Sizes. *Stat Med*. 2025 Aug;44(18-19):e70212.

computeEquipoise	<i>Compute fraction in equipoise</i>
------------------	--------------------------------------

Description

Compute fraction in equipoise

Usage

```
computeEquipoise(data, equipoiseBounds = c(0.3, 0.7))
```

Arguments

data	A data frame with at least the two columns described below.
equipoiseBounds	The bounds on the preference score to determine whether a subject is in equipoise.

Details

Computes the fraction of the population (the union of the target and comparator cohorts) who are in clinical equipoise (i.e. who had a reasonable chance of receiving either target or comparator, based on the baseline characteristics).

The data frame should have at least the following two columns:

- treatment (integer): Column indicating whether the person is in the target (1) or comparator (0) group
- propensityScore (numeric): Propensity score

Value

A numeric value (fraction in equipoise) between 0 and 1.

References

Walker AM, Patrick AR, Lauer MS, Hornbrook MC, Marin MG, Platt R, Roger VL, Stang P, and Schneeweiss S. (2013) A tool for assessing the feasibility of comparative effectiveness research, Comparative Effective Research, 3, 11-20

computeMdr	<i>Compute the minimum detectable relative risk</i>
------------	---

Description

Compute the minimum detectable relative risk

Usage

```
computeMdr(
  population,
  alpha = 0.05,
  power = 0.8,
  twoSided = TRUE,
  modelType = "cox"
)
```

Arguments

population	A data frame describing the study population as created using the createStudyPopulation function. This should at least have these columns: personSeqId, treatment, outcomeCount, timeAtRisk.
alpha	Type I error.
power	1 - beta, where beta is the type II error.
twoSided	Consider a two-sided test?
modelType	The type of outcome model that will be used. Possible values are "logistic", "poisson", or "cox". Currently only "cox" is supported.

Details

Compute the minimum detectable relative risk (MDRR) and expected standard error (SE) for a given study population, using the actual observed sample size and number of outcomes. Currently, only computations for Cox and logistic models are implemented. For Cox model, the computations by Schoenfeld (1983) is used. For logistic models Wald's z-test is used.

Value

A data frame with the MDRR and some counts.

References

Schoenfeld DA (1983) Sample-size formula for the proportional-hazards regression model, *Biometrics*, 39(3), 499-503

computePsAuc

Compute the area under the ROC curve

Description

Compute the area under the ROC curve of the propensity score.

Usage

```
computePsAuc(data, confidenceIntervals = FALSE, maxRows = 1e+05)
```

Arguments

data	A data frame with at least the two columns described below
confidenceIntervals	Compute 95 percent confidence intervals (computationally expensive for large data sets)
maxRows	Maximum number of rows to use. If the number of rows is larger, a random sample will be taken. This can increase speed, with minor cost to precision. Set to 0 to use all data.

Details

The data frame should have a least the following two columns:

- treatment (integer): Column indicating whether the person is in the target (1) or comparator (0) group.
- propensityScore (numeric): Propensity score.

Value

A tibble holding the AUC and its 95 percent confidence interval

Examples

```
treatment <- rep(0:1, each = 100)
propensityScore <- c(rnorm(100, mean = 0.4, sd = 0.25), rnorm(100, mean = 0.6, sd = 0.25))
data <- data.frame(treatment = treatment, propensityScore = propensityScore)
data <- data[data$propensityScore > 0 & data$propensityScore < 1, ]
computePsAuc(data)
```

convertUntypedListToCmAnalysesSpecifications

Convert untyped list to ScCsAnalysesSpecifications

Description

Convert untyped list to ScCsAnalysesSpecifications

Usage

```
convertUntypedListToCmAnalysesSpecifications(untypedList)
```

Arguments

untypedList	A list of untyped objects. For example, these could be objects from a call to <code>jsonlite::fromJSON()</code> . Importantly, <code>simplifyDataFrame</code> must be set to <code>FALSE</code> when doing so.
-------------	--

Value

An object of type `ScCsAnalysesSpecifications`.

```
createCmAnalysesSpecifications
    Create full CM analysis specifications
```

Description

Create full CM analysis specifications

Usage

```
createCmAnalysesSpecifications(
  cmAnalysisList,
  targetComparatorOutcomesList,
  analysesToExclude = NULL,
  refitPsForEveryOutcome = FALSE,
  refitPsForEveryStudyPopulation = TRUE,
  cmDiagnosticThresholds = createCmDiagnosticThresholds()
)
```

Arguments

cmAnalysisList A list of objects of type `cmAnalysis` as created using the [‘createCmAnalysis’](#) function.

targetComparatorOutcomesList A list of objects of type `targetComparatorOutcomes` as created using the [createTargetComparatorOutcomes](#) function.

analysesToExclude Analyses to exclude. See the Analyses to Exclude section for details.

refitPsForEveryOutcome Should the propensity model be fitted for every outcome (i.e. after people who already had the outcome are removed)? If false, a single propensity model will be fitted, and people who had the outcome previously will be removed afterwards.

refitPsForEveryStudyPopulation Should the propensity model be fitted for every study population definition? If false, a single propensity model will be fitted, and the study population criteria will be applied afterwards.

cmDiagnosticThresholds An object of type `CmDiagnosticThresholds` as created using [createCmDiagnosticThresholds\(\)](#).

Details

Analyses to Exclude:

Normally, `runCmAnalyses` will run all combinations of target-comparator-outcome-analyses settings. However, sometimes we may not need all those combinations. Using the `analysesToExclude` argument, we can remove certain items from the full matrix. This argument should be a data frame with at least one of the following columns:

- `targetId`
- `comparatorId`
- `nestingCohortId`

- outcomeId
- analysisId

This data frame will be joined to the outcome model reference table before executing, and matching rows will be removed. For example, if one specifies only one target ID and analysis ID, then any analyses with that target and that analysis ID will be skipped.

Value

An object of type CmAnalysesSpecifications.

createCmAnalysis	<i>Create a CohortMethod analysis specification</i>
------------------	---

Description

Create a CohortMethod analysis specification

Usage

```
createCmAnalysis(
  analysisId = 1,
  description = "",
  getDbCohortMethodDataArgs,
  createStudyPopulationArgs,
  createPsArgs = NULL,
  trimByPsArgs = NULL,
  truncateIptwArgs = NULL,
  matchOnPsArgs = NULL,
  stratifyByPsArgs = NULL,
  computeSharedCovariateBalanceArgs = NULL,
  computeCovariateBalanceArgs = NULL,
  fitOutcomeModelArgs = NULL
)
```

Arguments

analysisId	An integer that will be used later to refer to this specific set of analysis choices.
description	A short description of the analysis.
getDbCohortMethodDataArgs	An object representing the arguments to be used when calling the getDbCohortMethodData() function.
createStudyPopulationArgs	An object representing the arguments to be used when calling the createStudyPopulation() function.
createPsArgs	An object representing the arguments to be used when calling the createPs() function.
trimByPsArgs	An object representing the arguments to be used when calling the trimByPs() function.

truncateIptwArgs	An object representing the arguments to be used when calling the truncateIptw() function.
matchOnPsArgs	An object representing the arguments to be used when calling the matchOnPs() function.
stratifyByPsArgs	An object representing the arguments to be used when calling the stratifyByPs() function.
computeSharedCovariateBalanceArgs	An object representing the arguments to be used when calling the computeCovariateBalance() function per target-comparator-analysis.
computeCovariateBalanceArgs	An object representing the arguments to be used when calling the computeCovariateBalance() function per target-comparator-outcome-analysis.
fitOutcomeModelArgs	An object representing the arguments to be used when calling the fitOutcomeModel() function.

Details

Create a set of analysis choices, to be used with the [runCmAnalyses\(\)](#) function.

Providing a NULL value for any of the argument applies the corresponding step will not be executed. For example, if createPsArgs = NULL, no propensity scores will be computed.

Value

An object of type CmAnalysis, to be used with the [runCmAnalyses](#) function.

createCmDiagnosticThresholds
Create CohortMethod diagnostics thresholds

Description

Threshold used when calling [exportToCsv\(\)](#) to determine if we pass or fail diagnostics.

Usage

```
createCmDiagnosticThresholds(
  mdrThreshold = 10,
  easeThreshold = 0.25,
  sdmThreshold = 0.1,
  sdmAlpha = NULL,
  equipoiseThreshold = 0.2,
  generalizabilitySdmThreshold = 999
)
```

Arguments

mdrrThreshold	What is the maximum allowed minimum detectable relative risk (MDRR)?
easeThreshold	What is the maximum allowed expected absolute systematic error (EASE).
sdmThreshold	What is the maximum allowed standardized difference of mean (SDM)? If any covariate has an SDM exceeding this threshold, the diagnostic will fail.
sdmAlpha	What is the alpha for testing whether the absolute SDM exceeds sdmThreshold? If not provided, no significance testing will be performed and any absolute SDM greater than the threshold will be considered imbalance. Note that a Bonferroni adjustment will automatically be applied to adjust for the number of tests performed.
equipoiseThreshold	What is the minimum required equipoise?
generalizabilitySdmThreshold	What is the maximum allowed standardized difference of mean (SDM) when comparing the population before and after PS adjustments? If the SDM is greater than this value, the diagnostic will fail.

Details

The sdmThreshold and sdmAlpha arguments are independent of the threshold and alpha threshold provided to the createComputeCovariateBalanceArgs() function. The latter have no impact on blinding and diagnostics reported in the export.

Value

An object of type CmDiagnosticThresholds.

createCmTable1	<i>Create a table 1</i>
----------------	-------------------------

Description

Creates a formatted table of cohort characteristics, to be included in publications or reports.

Usage

```
createCmTable1(
  balance,
  specifications = getDefaultCmTable1Specifications(),
  beforeTargetPopSize = NULL,
  beforeComparatorPopSize = NULL,
  afterTargetPopSize = NULL,
  afterComparatorPopSize = NULL,
  beforeLabel = "Before matching",
  afterLabel = "After matching",
  targetLabel = "Target",
  comparatorLabel = "Comparator",
  percentDigits = 1,
  stdDiffDigits = 2
)
```

Arguments

balance	A data frame created by the computeCovariateBalance function.
specifications	Specifications of which covariates to display, and how.
beforeTargetPopSize	The number of people in the target cohort before matching/stratification/trimming, to mention in the table header. If not provide, no number will be included in the header.
beforeComparatorPopSize	The number of people in the comparator cohort before matching/stratification/trimming, to mention in the table header. If not provide, no number will be included in the header.
afterTargetPopSize	The number of people in the target cohort after matching/stratification/trimming, to mention in the table header. If not provide, no number will be included in the header.
afterComparatorPopSize	The number of people in the comparator cohort after matching/stratification/trimming, to mention in the table header. If not provide, no number will be included in the header.
beforeLabel	Label for identifying columns before matching / stratification / trimming.
afterLabel	Label for identifying columns after matching / stratification / trimming.
targetLabel	Label for identifying columns of the target cohort.
comparatorLabel	Label for identifying columns of the comparator cohort.
percentDigits	Number of digits to be used for percentages.
stdDiffDigits	Number of digits to be used for the standardized differences.

Value

A data frame with the formatted table 1.

```
createCohortMethodDataSimulationProfile
```

Create simulation profile

Description

Creates a profile based on the provided [CohortMethodData](#) object, which can be used to generate simulated data that has similar characteristics.

Usage

```
createCohortMethodDataSimulationProfile(cohortMethodData, minCellCount = 5)
```

Arguments

cohortMethodData	An object of type CohortMethodData as generated using getDbCohortMethodData() .
minCellCount	If > 0, will set to zero all low-prevalence covariates in the supplied simulation table in order to prevent identification of persons.

Details

The output of this function is an object that can be used by the [simulateCohortMethodData\(\)](#) function to generate a cohortMethodData object.

Value

An object of type CohortDataSimulationProfile.

```
createComputeCovariateBalanceArgs
```

*Create a parameter object for the function
[computeCovariateBalance\(\)](#)*

Description

Create a parameter object for the function [computeCovariateBalance\(\)](#)

Usage

```
createComputeCovariateBalanceArgs(  
  subgroupCovariateId = NULL,  
  maxCohortSize = 250000,  
  covariateFilter = NULL,  
  threshold = 0.1,  
  alpha = 0.05  
)
```

Arguments

subgroupCovariateId	Optional: a covariate ID of a binary covariate that indicates a subgroup of interest. Both the before and after populations will be restricted to this subgroup before computing covariate balance.
maxCohortSize	If the target or comparator cohort are larger than this number, they will be down-sampled before computing covariate balance to save time. Setting this number to 0 means no downsampling will be applied.
covariateFilter	Determines the covariates for which to compute covariate balance. Either a vector of covariate IDs, or a table 1 specifications object as generated for example using FeatureExtraction::getDefaultTable1Specifications() . If covariateFilter = NULL, balance will be computed for all variables found in the data.
threshold	Threshold value for the absolute value of the standardized difference of means (ASDM). If the ASDM exceeds this threshold it will be marked as unbalanced. (Hripcsak et al. 2025)
alpha	The family-wise alpha for testing whether the absolute value of the standardized difference of means is greater than the threshold. If not provided, any value greater than the threshold will be marked as unbalanced.

Details

Create an object defining the parameter values.

Value

An object of type `ComputeCovariateBalanceArgs`.

References

Hripcsak G, Zhang L, Chen Y, Li K, Suchard MA, Ryan PB, Schuemie MJ, Assessing Covariate Balance with Small Sample Sizes. *Statistics in Medicine* 44, no. 18-19 (2025): e70212

<code>createCreatePsArgs</code>	<i>Create a parameter object for the function <code>createPs()</code></i>
---------------------------------	---

Description

Create a parameter object for the function `createPs()`

Usage

```
createCreatePsArgs(
  excludeCovariateIds = c(),
  includeCovariateIds = c(),
  maxCohortSizeForFitting = 250000,
  errorOnHighCorrelation = TRUE,
  stopOnError = TRUE,
  prior = createPrior(priorType = "laplace", exclude = c(0), useCrossValidation = TRUE),
  control = createControl(noiseLevel = "silent", cvType = "auto", seed = 1,
    resetCoefficients = TRUE, tolerance = 2e-07, cvRepetitions = 10, startingVariance =
    0.01),
  estimator = "att"
)
```

Arguments

<code>excludeCovariateIds</code>	Exclude these covariates from the propensity model.
<code>includeCovariateIds</code>	Include only these covariates in the propensity model.
<code>maxCohortSizeForFitting</code>	If the target or comparator cohort are larger than this number, they will be down-sampled before fitting the propensity model. The model will be used to compute propensity scores for all subjects. The purpose of the sampling is to gain speed. Setting this number to 0 means no downsampling will be applied.
<code>errorOnHighCorrelation</code>	If true, the function will test each covariate for correlation with the treatment assignment. If any covariate has an unusually high correlation (either positive or negative), this will throw an error.
<code>stopOnError</code>	If an error occurs, should the function stop? Else, the two cohorts will be assumed to be perfectly separable.

prior	The prior used to fit the model. See Cyclops::createPrior() for details.
control	The control object used to control the cross-validation used to determine the hyperparameters of the prior (if applicable). See Cyclops::createControl() for details.
estimator	The type of estimator for the IPTW. Options are estimator = "ate" for the average treatment effect, estimator = "att" for the average treatment effect in the treated, and estimator = "ato" for the average treatment effect in the overlap population.

Details

Create an object defining the parameter values.

Value

An object of type CreatePsArgs.

createCreateStudyPopulationArgs

Create a parameter object for the function
[createStudyPopulation\(\)](#)

Description

Create a parameter object for the function [createStudyPopulation\(\)](#)

Usage

```
createCreateStudyPopulationArgs(
  removeSubjectsWithPriorOutcome = TRUE,
  priorOutcomeLookback = 99999,
  minDaysAtRisk = 1,
  maxDaysAtRisk = 99999,
  riskWindowStart = 0,
  startAnchor = "cohort start",
  riskWindowEnd = 0,
  endAnchor = "cohort end",
  censorAtNewRiskWindow = FALSE
)
```

Arguments

removeSubjectsWithPriorOutcome	Remove subjects that have the outcome prior to the risk window start?
priorOutcomeLookback	How many days should we look back when identifying prior outcomes?
minDaysAtRisk	The minimum required number of days at risk. Risk windows with fewer days than this number are removed from the analysis.
maxDaysAtRisk	The maximum allowed number of days at risk. Risk windows that are longer will be truncated to this number of days.

riskWindowStart	The start of the risk window (in days) relative to the startAnchor.
startAnchor	The anchor point for the start of the risk window. Can be "cohort start" or "cohort end".
riskWindowEnd	The end of the risk window (in days) relative to the endAnchor.
endAnchor	The anchor point for the end of the risk window. Can be "cohort start" or "cohort end".
censorAtNewRiskWindow	If a subject is in multiple cohorts, should time-at-risk be censored when the new time-at-risk starts to prevent overlap?

Details

Create an object defining the parameter values.

Value

An object of type CreateStudyPopulationArgs.

```
createDefaultMultiThreadingSettings
```

Create default CohortMethod multi-threading settings

Description

Create CohortMethod multi-threading settings based on the maximum number of cores to be used.

Usage

```
createDefaultMultiThreadingSettings(maxCores)
```

Arguments

maxCores Maximum number of CPU cores to use.

Value

An object of type CmMultiThreadingSettings.

See Also

[createMultiThreadingSettings\(\)](#)

Examples

```
settings <- createDefaultMultiThreadingSettings(10)
```

```
createFitOutcomeModelArgs
```

Create a parameter object for the function `fitOutcomeModel()`

Description

Create a parameter object for the function `fitOutcomeModel()`

Usage

```
createFitOutcomeModelArgs(
  modelType = "cox",
  stratified = FALSE,
  useCovariates = FALSE,
  inversePtWeighting = FALSE,
  bootstrapCi = FALSE,
  bootstrapReplicates = 200,
  interactionCovariateIds = c(),
  excludeCovariateIds = c(),
  includeCovariateIds = c(),
  profileGrid = NULL,
  profileBounds = c(log(0.1), log(10)),
  prior = createPrior(priorType = "laplace", useCrossValidation = TRUE),
  control = createControl(cvType = "auto", seed = 1, resetCoefficients = TRUE,
    startingVariance = 0.01, tolerance = 2e-07, cvRepetitions = 10, noiseLevel = "quiet")
)
```

Arguments

<code>modelType</code>	The type of outcome model that will be used. Possible values are "logistic", "poisson", or "cox".
<code>stratified</code>	Should the regression be conditioned on the strata defined in the population object (e.g. by matching or stratifying on propensity scores)?
<code>useCovariates</code>	Whether to use the covariates in the cohortMethodData object in the outcome model.
<code>inversePtWeighting</code>	Use inverse probability of treatment weighting (IPTW)
<code>bootstrapCi</code>	Compute confidence interval using bootstrapping instead of likelihood profiling?
<code>bootstrapReplicates</code>	When using bootstrapping to compute confidence intervals, how many replicates should be sampled?
<code>interactionCovariateIds</code>	An optional vector of covariate IDs to use to estimate interactions with the main treatment effect.
<code>excludeCovariateIds</code>	Exclude these covariates from the outcome model.
<code>includeCovariateIds</code>	Include only these covariates in the outcome model.

profileGrid	A one-dimensional grid of points on the log(relative risk) scale where the likelihood for coefficient of variables is sampled. See details.
profileBounds	The bounds (on the log relative risk scale) for the adaptive sampling of the likelihood function. See details.
prior	The prior used to fit the model. See Cyclops::createPrior() for details. The prior is only applied to non-treatment variables, so is not used when useCovariates = FALSE.
control	The control object used to control the cross-validation used to determine the hyperparameters of the prior (if applicable). See Cyclops::createControl() for details.

Details

Create an object defining the parameter values.

For likelihood profiling, either specify the profileGrid for a completely user- defined grid, or profileBounds for an adaptive grid. Both should be defined on the log effect size scale. When both profileGrid and profileGrid are NULL likelihood profiling is disabled.

Value

An object of type ComputeCovariateBalanceArgs.

createGetDbCohortMethodDataArgs

Create a parameter object for the function
[getDbCohortMethodData\(\)](#)

Description

Create a parameter object for the function [getDbCohortMethodData\(\)](#)

Usage

```
createGetDbCohortMethodDataArgs(
  removeDuplicateSubjects = "keep first, truncate to second",
  firstExposureOnly = TRUE,
  washoutPeriod = 365,
  nestingCohortId = NULL,
  restrictToCommonPeriod = TRUE,
  minAge = NULL,
  maxAge = NULL,
  genderConceptIds = NULL,
  studyStartDate = "",
  studyEndDate = "",
  maxCohortSize = 0,
  covariateSettings
)
```

Arguments

<code>removeDuplicateSubjects</code>	Remove subjects that are in both the target and comparator cohort? See details for allowed values. Note that this is typically done in the <code>createStudyPopulation</code> function, but can already be done here for efficiency reasons.
<code>firstExposureOnly</code>	Should only the first exposure per subject be included? Note that this is typically done in the <code>createStudyPopulation()</code> function, but can already be done here for efficiency reasons.
<code>washoutPeriod</code>	The minimum required continuous observation time prior to index date for a person to be included in the cohort. Note that this is typically done in the <code>createStudyPopulation</code> function, but can already be done here for efficiency reasons.
<code>nestingCohortId</code>	A cohort definition ID identifying the records in the <code>nestingCohortTable</code> to use as nesting cohort.
<code>restrictToCommonPeriod</code>	Restrict the analysis to the period when both treatments are observed?
<code>minAge</code>	Minimum age at index date at which patient time will be included in the analysis. If not specified, no minimum age restriction will be applied.
<code>maxAge</code>	Maximum age at index date at which patient time will be included in the analysis. If not specified, no maximum age restriction will be applied.
<code>genderConceptIds</code>	Set of gender concept IDs to restrict the population to. If not specified, no restriction on gender will be applied.
<code>studyStartDate</code>	A calendar date specifying the minimum date that a cohort index date can appear. Date format is 'yyyymmdd'.
<code>studyEndDate</code>	A calendar date specifying the maximum date that a cohort index date can appear. Date format is 'yyyymmdd'. Important: the study end data is also used to truncate risk windows, meaning no outcomes beyond the study end date will be considered.
<code>maxCohortSize</code>	If either the target or the comparator cohort is larger than this number it will be sampled to this size. <code>maxCohortSize = 0</code> indicates no maximum size.
<code>covariateSettings</code>	An object of type <code>covariateSettings</code> as created using the <code>FeatureExtraction::createCovariateSettings()</code> function, or a list of covariate settings objects.

Details

Create an object defining the parameter values.

The `removeDuplicateSubjects` argument can have one of the following values:

- "keep first, truncate to second": When a subjects appear in both target and comparator cohort, only keep whichever cohort is first in time. If the other cohort starts before the first has ended, the first cohort will be truncated to stop the day before the second starts. If both cohorts start simultaneous, the person is removed from the analysis.
- "keep first": When a subjects appear in both target and comparator cohort, only keep whichever cohort is first in time. If both cohorts start simultaneous, the person is removed from the analysis.
- "remove all": Remove subjects that appear in both target and comparator cohort completely from the analysis."
- "keep all": Do not remove subjects that appear in both target and comparator cohort

Value

An object of type `GetDbCohortMethodDataArgs`.

`createMatchOnPsArgs` *Create a parameter object for the function `matchOnPs()`*

Description

Create a parameter object for the function `matchOnPs()`

Usage

```
createMatchOnPsArgs(
  caliper = 0.2,
  caliperScale = "standardized logit",
  maxRatio = 1,
  allowReverseMatch = FALSE,
  matchColumns = c(),
  matchCovariateIds = c()
)
```

Arguments

<code>caliper</code>	The caliper for matching. A caliper is the distance which is acceptable for any match. Observations which are outside of the caliper are dropped. A caliper of 0 means no caliper is used.
<code>caliperScale</code>	The scale on which the caliper is defined. Three scales are supported: <code>caliperScale = 'propensity score'</code> , <code>caliperScale = 'standardized'</code> , or <code>caliperScale = 'standardized logit'</code> . On the standardized scale, the caliper is interpreted in standard deviations of the propensity score distribution. 'standardized logit' is similar, except that the propensity score is transformed to the logit scale because the PS is more likely to be normally distributed on that scale (Austin, 2011).
<code>maxRatio</code>	The maximum number of persons in the comparator arm to be matched to each person in the treatment arm. A <code>maxRatio</code> of 0 means no maximum: all comparators will be assigned to a target person.
<code>allowReverseMatch</code>	Allows n-to-1 matching if target arm is larger
<code>matchColumns</code>	Names or numbers of one or more columns in the data <code>data.frame</code> on which subjects should be stratified prior to matching. No persons will be matched with persons outside of the strata identified by the values in these columns.
<code>matchCovariateIds</code>	One or more covariate IDs in the <code>cohortMethodData</code> object on which subjects should be also matched.

Details

Create an object defining the parameter values.

Value

An object of type `MatchOnPsArgs`.

References

Austin, PC. (2011) Optimal caliper widths for propensity-score matching when estimating differences in means and differences in proportions in observational studies, *Pharmaceutical statistics*, March, 10(2):150-161.

createMultiThreadingSettings

Create CohortMethod multi-threading settings

Description

Create CohortMethod multi-threading settings

Usage

```
createMultiThreadingSettings(
  getDbCohortMethodDataThreads = 1,
  createPsThreads = 1,
  psCvThreads = 1,
  createStudyPopThreads = 1,
  trimMatchStratifyThreads = 1,
  computeSharedBalanceThreads = 1,
  computeBalanceThreads = 1,
  prefilterCovariatesThreads = 1,
  fitOutcomeModelThreads = 1,
  outcomeCvThreads = 1,
  calibrationThreads = 1
)
```

Arguments

getDbCohortMethodDataThreads	The number of parallel threads to use for building the cohortMethod data objects.
createPsThreads	The number of parallel threads to use for fitting the propensity models.
psCvThreads	The number of parallel threads to use for the cross- validation when estimating the hyperparameter for the propensity model. Note that the total number of CV threads at one time could be createPsThreads * psCvThreads.
createStudyPopThreads	The number of parallel threads to use for creating the study population.
trimMatchStratifyThreads	The number of parallel threads to use for trimming, matching and stratifying.
computeSharedBalanceThreads	The number of parallel threads to use for computing shared covariate balance.
computeBalanceThreads	The number of parallel threads to use for computing covariate balance.
prefilterCovariatesThreads	The number of parallel threads to use for prefiltering covariates.

fitOutcomeModelThreads

The number of parallel threads to use for fitting the outcome models.

outcomeCvThreads

The number of parallel threads to use for the cross- validation when estimating the hyperparameter for the outcome model. Note that the total number of CV threads at one time could be fitOutcomeModelThreads * outcomeCvThreads.

calibrationThreads

The number of parallel threads to use for empirical calibration.

Value

An object of type CmMultiThreadingSettings.

See Also

[createDefaultMultiThreadingSettings\(\)](#)

createOutcome	<i>Create outcome definition</i>
---------------	----------------------------------

Description

Create outcome definition

Usage

```
createOutcome(
  outcomeId,
  outcomeOfInterest = TRUE,
  trueEffectSize = NA,
  priorOutcomeLookback = NULL,
  riskWindowStart = NULL,
  startAnchor = NULL,
  riskWindowEnd = NULL,
  endAnchor = NULL
)
```

Arguments

outcomeId An integer used to identify the outcome in the outcome cohort table.

outcomeOfInterest

Is this an outcome of interest? If not, creation of non-essential files will be skipped, including outcome-specific covariate balance files. This could be helpful to speed up analyses with many controls, for which we're only interested in the effect size estimate.

trueEffectSize For negative and positive controls: the known true effect size. To be used for empirical calibration. Negative controls have trueEffectSize = 1. If the true effect size is unknown, use trueEffectSize = NA

priorOutcomeLookback

How many days should we look back when identifying prior. outcomes?

riskWindowStart	The start of the risk window (in days) relative to the startAnchor.
startAnchor	The anchor point for the start of the risk window. Can be "cohort start" or "cohort end".
riskWindowEnd	The end of the risk window (in days) relative to the endAnchor.
endAnchor	The anchor point for the end of the risk window. Can be "cohort start" or "cohort end".

Details

Any settings here that are not NULL will override any values set in `createCreateStudyPopulationArgs()`.

Value

An object of type Outcome, to be used in `createTargetComparatorOutcomes()`.

createPs	<i>Create propensity scores</i>
----------	---------------------------------

Description

Creates propensity scores and inverse probability of treatment weights (IPTW) using a regularized logistic regression.

Usage

```
createPs(
  cohortMethodData,
  population = NULL,
  createPsArgs = createCreatePsArgs()
)
```

Arguments

cohortMethodData	An object of type <code>CohortMethodData</code> as generated using <code>getDbCohortMethodData()</code> .
population	A data frame describing the population. This should at least have a rowId column corresponding to the rowId column in the <code>CohortMethodData</code> covariates object and a treatment column. If population is not specified, the full population in the <code>CohortMethodData</code> will be used.
createPsArgs	And object of type <code>CreatePsArgs</code> as created by the <code>createCreatePsArgs()</code> function

Details

IPTW estimates either the average treatment effect (ate) or average treatment effect in the treated (att) using stabilized inverse propensity scores (Xu et al. 2010).

References

Xu S, Ross C, Raebel MA, Shetterly S, Blanchette C, Smith D. Use of stabilized inverse propensity scores as weights to directly estimate relative risk and its confidence intervals. Value Health. 2010;13(2):273-277. doi:10.1111/j.1524-4733.2009.00671.x

Examples

```
data(cohortMethodDataSimulationProfile)
cohortMethodData <- simulateCohortMethodData(cohortMethodDataSimulationProfile, n = 100)
ps <- createPs(cohortMethodData, createPsArgs = createCreatePsArgs())
```

```
createResultsDataModel
```

Create the results data model tables on a database server.

Description

Create the results data model tables on a database server.

Usage

```
createResultsDataModel(
  connectionDetails = NULL,
  databaseSchema,
  tablePrefix = ""
)
```

Arguments

connectionDetails

DatabaseConnector connectionDetails instance @seealso [DatabaseConnector::createConnectionDetails](#)

databaseSchema The schema on the server where the tables will be created.

tablePrefix (Optional) string to insert before table names for database table names

Details

Only PostgreSQL and SQLite servers are supported.

```
createStratifyByPsArgs
```

Create a parameter object for the function [stratifyByPs\(\)](#)

Description

Create a parameter object for the function [stratifyByPs\(\)](#)

Usage

```
createStratifyByPsArgs(
  numberOfStrata = 10,
  baseSelection = "all",
  stratificationColumns = c(),
  stratificationCovariateIds = c()
)
```

Arguments

<code>numberOfStrata</code>	How many strata? The boundaries of the strata are automatically defined to contain equal numbers of target persons.
<code>baseSelection</code>	What is the base selection of subjects where the strata bounds are to be determined? Strata are defined as equally-sized strata inside this selection. Possible values are "all", "target", and "comparator".
<code>stratificationColumns</code>	Names or numbers of one or more columns in the data <code>data.frame</code> on which subjects should be stratified prior to matching. No persons will be matched with persons outside of the strata identified by the values in these columns.
<code>stratificationCovariateIds</code>	One or more covariate IDs in the <code>cohortMethodData</code> object on which subjects should also be stratified.

Details

Create an object defining the parameter values.

Value

An object of type `StratifyByPsArgs`.

```
createStudyPopulation
```

Create a study population

Description

Create a study population

Usage

```
createStudyPopulation(
  cohortMethodData,
  population = NULL,
  outcomeId = NULL,
  createStudyPopulationArgs = createCreateStudyPopulationArgs()
)
```

Arguments

cohortMethodData

An object of type [CohortMethodData](#) as generated using [getDbCohortMethodData\(\)](#).

population

If specified, this population will be used as the starting point instead of the cohorts in the cohortMethodData object.

outcomeId

The ID of the outcome. If NULL, no outcome-specific transformations will be performed.

createStudyPopulationArgs

An object of type CreateStudyPopulationArgs as created by the [createCreateStudyPopulationArgs\(\)](#) function.

Details

Create a study population by enforcing certain inclusion and exclusion criteria, defining a risk window, and determining which outcomes fall inside the risk window.

Value

A tibble specifying the study population. This tibble will have the following columns:

- `rowId`: A unique identifier for an exposure.
- `personSeqId`: The person sequence ID of the subject.
- `cohortStartDate`: The index date.
- `outcomeCount`: The number of outcomes observed during the risk window.
- `timeAtRisk`: The number of days in the risk window.
- `survivalTime`: The number of days until either the outcome or the end of the risk window.

createTargetComparatorOutcomes

Create target-comparator-outcomes combinations.

Description

Create target-comparator-outcomes combinations.

Usage

```
createTargetComparatorOutcomes(
  targetId,
  comparatorId,
  outcomes,
  nestingCohortId = NULL,
  excludedCovariateConceptIds = c(),
  includedCovariateConceptIds = c()
)
```

Arguments

targetId A cohort ID identifying the target exposure in the exposure table.

comparatorId A cohort ID identifying the comparator exposure in the exposure table.

outcomes A list of object of type Outcome as created by [createOutcome\(\)](#).

nestingCohortId (Optional) the nesting cohort ID. If provided, this will override the nesting cohort ID used in [createGetDbCohortMethodDataArgs\(\)](#).

excludedCovariateConceptIds A list of concept IDs that cannot be used to construct covariates. This argument is to be used only for exclusion concepts that are specific to the target-comparator combination.

includedCovariateConceptIds A list of concept IDs that must be used to construct covariates. This argument is to be used only for inclusion concepts that are specific to the target-comparator combination.

Details

Create a set of hypotheses of interest, to be used with the [runCmAnalyses\(\)](#) function.

Value

An object of type TargetComparatorOutcomes.

createTrimByPsArgs	<i>Create a parameter object for the function trimByPs()</i>
--------------------	--

Description

Create a parameter object for the function [trimByPs\(\)](#)

Usage

```
createTrimByPsArgs(
  trimFraction = NULL,
  equipoiseBounds = NULL,
  maxWeight = NULL,
  trimMethod = "symmetric"
)
```

Arguments

trimFraction	For trimFraction = symmetric: the PS cut-off value. For trimFraction = asymmetric or reverse asymmetric: the fraction that will be removed from each treatment group. See trimMethod for more details.
equipoiseBounds	A 2-dimensional numeric vector containing the upper and lower bound on the preference score (Walker, 2013) for keeping persons.
maxWeight	The maximum allowed IPTW.
trimMethod	The trimming method to be performed. Three methods are supported: • symmetric: trims all units with estimated PS outside the interval (trimFraction, 1-trimFraction), following Crump et al. (2009). • asymmetric: removes all units not in the overlap PS range and trims the trimFraction target persons with the lowest propensity scores and comparator persons with the highest propensity scores, following Sturmer et al. (2010). • reverse asymmetric: removes all units not in the overlap PS range and trims the trimFraction target persons with the highest propensity scores and comparator persons with the lowest propensity scores (not suggested).

Details

Create an object defining the parameter values. Set any argument to NULL to not use it for trimming.

Value

An object of type TrimByPsArgs.

References

- Walker AM, Patrick AR, Lauer MS, Hornbrook MC, Marin MG, Platt R, Roger VL, Stang P, and Schneeweiss S. (2013) A tool for assessing the feasibility of comparative effectiveness research, Comparative Effective Research, 3, 11-20
- Crump, Richard K., V. Joseph Hotz, Guido W. Imbens, and Oscar A. Mitnik. 2009. Dealing with limited overlap in estimation of average treatment effects. Biometrika 96(1): 187-199.
- Sturmer T, Rothman KJ, Avorn J, Glynn RJ. Treatment effects in the presence of unmeasured confounding: dealing with observations in the tails of the propensity score distribution—a simulation study. Am J Epidemiol. 2010 Oct 1;172(7):843-54.

```
createTruncateIptwArgs
```

Create a parameter object for the function [truncateIptw\(\)](#)

Description

Create a parameter object for the function [truncateIptw\(\)](#)

Usage

```
createTruncateIptwArgs(maxWeight = 10)
```

Arguments

maxWeight The maximum allowed IPTW.

Details

Create an object defining the parameter values.

Value

An object of type TruncateIptwArgs.

drawAttritionDiagram *Draw the attrition diagram*

Description

drawAttritionDiagram draws the attrition diagram, showing how many people were excluded from the study population, and for what reasons.

Usage

```
drawAttritionDiagram(
  object,
  targetLabel = "Target",
  comparatorLabel = "Comparator",
  fileName = NULL
)
```

Arguments

object Either an object of type cohortMethodData, a population object generated by functions like createStudyPopulation, or an object of type outcomeModel.

targetLabel A label to us for the target cohort.

comparatorLabel A label to us for the comparator cohort.

fileName Name of the file where the plot should be saved, for example 'plot.png'. See the function ggsave in the ggplot2 package for supported file formats.

Value

A ggplot object. Use the [ggsave](#) function to save to file in a different format.

exportToCsv	<i>Export cohort method results to CSV files</i>
-------------	--

Description

Export cohort method results to CSV files

Usage

```
exportToCsv(
  outputFolder,
  exportFolder = file.path(outputFolder, "export"),
  databaseId,
  minCellCount = 5,
  maxCores = 1
)
```

Arguments

outputFolder	The folder where runCmAnalyses() generated all results.
exportFolder	The folder where the CSV files will written.
databaseId	A unique ID for the database. This will be appended to most tables.
minCellCount	To preserve privacy: the minimum number of subjects contributing to a count before it can be included in the results. If the count is below this threshold, it will be set to -minCellCount.
maxCores	How many parallel cores should be used?

Details

This requires that `runCmAnalyses()` has been executed first. It exports all the results in the outputFolder to CSV files for sharing with other sites.

Value

Does not return anything. Is called for the side-effect of populating the exportFolder with CSV files.

fitOutcomeModel	<i>Create an outcome model, and compute the relative risk</i>
-----------------	---

Description

Create an outcome model, and computes the relative risk

Usage

```
fitOutcomeModel(
  population,
  cohortMethodData = NULL,
  fitOutcomeModelArgs = createFitOutcomeModelArgs()
)
```

Arguments

- population** A population object generated by `createStudyPopulation()`, potentially filtered by other functions.
- cohortMethodData** An object of type `CohortMethodData` as generated using `getDbCohortMethodData()`. Can be omitted if not using covariates and not using interaction terms.
- fitOutcomeModelArgs** An object of type `FitOutcomeModelArgs` as generated using the `createFitOutcomeModelArgs()` function.

Details

For likelihood profiling, either specify the `profileGrid` for a completely user- defined grid, or `profileBounds` for an adaptive grid. Both should be defined on the log effect size scale. When both `profileGrid` and `profileGrid` are NULL likelihood profiling is disabled.

Value

An object of class `OutcomeModel`. Generic function `print`, `coef`, and `confint` are available.

getAttritionTable	<i>Get the attrition table for a population</i>
-------------------	---

Description

Get the attrition table for a population

Usage

```
getAttritionTable(object)
```

Arguments

- object** Either an object of type `CohortMethodData`, a population object generated by functions like `createStudyPopulation()`, or an object of type `OutcomeModel`.

Value

A tibble specifying the number of people and exposures in the population after specific steps of filtering.

getDataMigrator	<i>Get database migrations instance</i>
-----------------	---

Description

Returns ResultModelManager DataMigrationsManager instance.

Usage

```
getDataMigrator(connectionDetails, databaseSchema, tablePrefix = "")
```

Arguments

connectionDetails	DatabaseConnector connection details object
databaseSchema	String schema where database schema lives
tablePrefix	(Optional) Use if a table prefix is used before table names (e.g. "cd_")

Value

Instance of ResultModelManager::DataMigrationManager that has interface for converting existing data models

getDbCohortMethodData	<i>Get the cohort data from the server</i>
-----------------------	--

Description

This function executes a large set of SQL statements against the database in OMOP CDM format to extract the data needed to perform the analysis.

Usage

```
getDbCohortMethodData(
  connectionDetails,
  cdmDatabaseSchema,
  tempEmulationSchema = getOption("sqlRenderTempEmulationSchema"),
  targetId,
  comparatorId,
  outcomeIds,
  exposureDatabaseSchema = cdmDatabaseSchema,
  exposureTable = "drug_era",
  outcomeDatabaseSchema = cdmDatabaseSchema,
  outcomeTable = "condition_occurrence",
  nestingCohortDatabaseSchema = cdmDatabaseSchema,
  nestingCohortTable = "cohort",
  getDbCohortMethodDataArgs = createGetDbCohortMethodDataArgs()
)
```

Arguments

connectionDetails

An R object of type connectionDetails created using the [DatabaseConnector::createConnection](#) function.

cdmDatabaseSchema

The name of the database schema that contains the OMOP CDM instance. Requires read permissions to this database. On SQL Server, this should specify both the database and the schema, so for example 'cdm_instance.dbo'.

tempEmulationSchema

Some database platforms like Oracle and Impala do not truly support temp tables. To emulate temp tables, provide a schema with write privileges where temp tables can be created.

targetId

A unique identifier to define the target cohort. If exposureTable = DRUG_ERA, targetId is a concept ID and all descendant concepts within that concept ID will be used to define the cohort. If exposureTable <> DRUG_ERA, targetId is used to select the COHORT_DEFINITION_ID in the cohort-like table.

comparatorId

A unique identifier to define the comparator cohort. If exposureTable = DRUG_ERA, comparatorId is a concept ID and all descendant concepts within that concept ID will be used to define the cohort. If exposureTable <> DRUG_ERA, comparatorId is used to select the COHORT_DEFINITION_ID in the cohort-like table.

outcomeIds

A list of cohort IDs used to define outcomes.

exposureDatabaseSchema

The name of the database schema that is the location where the exposure data used to define the exposure cohorts is available.

exposureTable

The tablename that contains the exposure cohorts. If exposureTable <> DRUG_ERA, then expectation is exposureTable has format of COHORT table: COHORT_DEFINITION_ID, SUBJECT_ID, COHORT_START_DATE, COHORT_END_DATE.

outcomeDatabaseSchema

The name of the database schema that is the location where the data used to define the outcome cohorts is available.

outcomeTable

The tablename that contains the outcome cohorts. If outcomeTable <> CONDITION_OCCURRENCE, then expectation is outcomeTable has format of COHORT table: COHORT_DEFINITION_ID, SUBJECT_ID, COHORT_START_DATE, COHORT_END_DATE.

nestingCohortDatabaseSchema

The name of the database schema that is the location where the data used to define the nesting cohorts is available.

nestingCohortTable

The tablename that contains the nesting cohorts. Must have the format of COHORT table: COHORT_DEFINITION_ID, SUBJECT_ID, COHORT_START_DATE, COHORT_END_DATE.

getDbCohortMethodDataArgs

An object of type GetDbCohortMethodDataArgs as created by the [createGetDbCohortMethodData](#) function.

Details

Based on the arguments, the treatment and comparator cohorts are retrieved, as well as outcomes occurring in exposed subjects. The treatment and comparator cohorts can be identified using the DRUG_ERA table, or through user-defined cohorts in a cohort table either inside the CDM schema

or in a separate schema. Similarly, outcomes are identified using the CONDITION_ERA table or through user-defined cohorts in a cohort table either inside the CDM schema or in a separate schema. Optionally, the target and comparator cohorts can be restricted to be within a nesting cohort, which can reside in a different database schema and table.

Value

A [CohortMethodData](#) object.

```
getDefaultCmTable1Specifications
```

Get the default table 1 specifications

Description

Loads the default specifications for a table 1, to be used with the [createCmTable1](#) function.
Important: currently only works for binary covariates.

Usage

```
getDefaultCmTable1Specifications()
```

Value

A specifications objects.

```
getDiagnosticsSummary
```

Get a summary report of the analyses diagnostics

Description

Get a summary report of the analyses diagnostics

Usage

```
getDiagnosticsSummary(outputFolder)
```

Arguments

`outputFolder` Name of the folder where all the outputs have been written to.

Value

A tibble containing summary diagnostics for each outcome-covariate-analysis combination.

getFileReference	<i>Get file reference</i>
------------------	---------------------------

Description

Get file reference

Usage

```
getFileReference(outputFolder)
```

Arguments

outputFolder Name of the folder where all the outputs have been written to.

Value

A tibble containing file names of artifacts generated for each target-comparator-outcome-analysis combination.

getFollowUpDistribution	<i>Get the distribution of follow-up time</i>
-------------------------	---

Description

Get the distribution of follow-up time

Usage

```
getFollowUpDistribution(population, quantiles = c(0, 0.25, 0.5, 0.75, 1))
```

Arguments

population A data frame describing the study population as created using the [createStudyPopulation](#) function. This should at least have these columns: treatment, timeAtRisk.

quantiles The quantiles of the population to compute minimum follow-up time for.

Details

Get the distribution of follow-up time as quantiles. Follow-up time is defined as time-at-risk, so not censored at the outcome.

Value

A data frame with per treatment group at each quantile the amount of follow-up time available.

getGeneralizabilityTable

Get information on generalizability

Description

to assess generalizability we compare the distribution of covariates before and after any (propensity score) adjustments. We compute the standardized difference of mean as our metric of generalizability. (Lipton et al., 2017)

Depending on our target estimand, we need to consider a different base population for generalizability. For example, if we aim to estimate the average treatment effect in the treated (ATT), our base population should be the target population, meaning we should consider the covariate distribution before and after PS adjustment in the target population only. By default this function will attempt to select the right base population based on what operations have been performed on the population. For example, if PS matching has been performed we assume the target estimand is the ATT, and the target population is selected as base.

Requires running `computeCovariateBalance()` first.

Usage

```
getGeneralizabilityTable(balance, baseSelection = "auto")
```

Arguments

<code>balance</code>	A data frame created by the <code>computeCovariateBalance</code> function.
<code>baseSelection</code>	The selection of the population to consider for generalizability. Options are "auto", "target", "comparator", and "both". The "auto" option will attempt to use the balance meta-data to pick the most appropriate population based on the target estimator.

Value

A tibble with the following columns:

- `covariateId`: The ID of the covariate. Can be linked to the `covariates` and `covariateRef` tables in the `CohortMethodData` object.
- `covariateName`: The name of the covariate.
- `beforeMatchingMean`: The mean covariate value before any (propensity score) adjustment.
- `afterMatchingMean`: The mean covariate value after any (propensity score) adjustment.
- `stdDiff`: The standardized difference of means between before and after adjustment.

The tibble also has a 'baseSelection' attribute, documenting the base population used to assess generalizability.

References

Tipton E, Hallberg K, Hedges LV, Chan W (2017) Implications of Small Samples for Generalization: Adjustments and Rules of Thumb, *Eval Rev.* Oct;41(5):472-505.

```
getInteractionResultsSummary
```

Get a summary report of the analyses results

Description

Get a summary report of the analyses results

Usage

```
getInteractionResultsSummary(outputFolder)
```

Arguments

`outputFolder` Name of the folder where all the outputs have been written to.

Value

A tibble containing summary statistics for each target-comparator-outcome-analysis combination.

```
getOutcomeModel
```

Get the outcome model

Description

Get the full outcome model, so showing the betas of all variables included in the outcome model, not just the treatment variable.

Usage

```
getOutcomeModel(outcomeModel, cohortMethodData)
```

Arguments

`outcomeModel` An object of type `OutcomeModel` as generated using the `fitOutcomeModel()` function.

`cohortMethodData` An object of type `CohortMethodData` as generated using `getDbCohortMethodData()`.

Value

A tibble.

getPsModel	<i>Get the propensity model</i>
------------	---------------------------------

Description

Returns the coefficients and names of the covariates with non-zero coefficients.

Usage

```
getPsModel(propensityScore, cohortMethodData)
```

Arguments

propensityScore

The propensity scores as generated using the [createPs\(\)](#) function.

cohortMethodData

An object of type [CohortMethodData](#) as generated using [getDbCohortMethodData\(\)](#).

Value

A tibble.

getResultsDataModelSpecifications	<i>Get specifications for CohortMethod results data model</i>
-----------------------------------	---

Description

Get specifications for CohortMethod results data model

Usage

```
getResultsDataModelSpecifications()
```

Value

A tibble data frame object with specifications

getResultsSummary	<i>Get a summary report of the analyses results</i>
-------------------	---

Description

Get a summary report of the analyses results

Usage

```
getResultsSummary(outputFolder)
```

Arguments

outputFolder Name of the folder where all the outputs have been written to.

Value

A tibble containing summary statistics for each target-comparator-outcome-analysis combination.

isCohortMethodData	<i>Check whether an object is a CohortMethodData object</i>
--------------------	---

Description

Check whether an object is a CohortMethodData object

Usage

```
isCohortMethodData(x)
```

Arguments

x The object to check.

Value

A logical value.

loadCmAnalysisList	<i>Load a list of CmAnalysis from file</i>
--------------------	--

Description

Load a list of objects of type CmAnalysis from file. The file is in JSON format.

Usage

```
loadCmAnalysisList(file)
```

Arguments

file	The name of the file
------	----------------------

Value

A list of objects of type CmAnalysis.

loadCohortMethodData	<i>Load the cohort method data from a file</i>
----------------------	--

Description

Loads an object of type [CohortMethodData](#) from a file in the file system.

Usage

```
loadCohortMethodData(file)
```

Arguments

file	The name of the file containing the data.
------	---

Value

An object of class [CohortMethodData](#).

```
loadTargetComparatorOutcomesList
```

Load a list of TargetComparatorOutcomes from file

Description

Load a list of objects of type TargetComparatorOutcomes from file. The file is in JSON format.

Usage

```
loadTargetComparatorOutcomesList(file)
```

Arguments

file	The name of the file
------	----------------------

Value

A list of objects of type TargetComparatorOutcomes.

```
matchOnPs
```

Match persons by propensity score

Description

Use the provided propensity scores to match target to comparator persons.

Usage

```
matchOnPs(
  population,
  matchOnPsArgs = createMatchOnPsArgs(),
  cohortMethodData = NULL
)
```

Arguments

population	A data frame with the three columns described below.
matchOnPsArgs	An object of type MatchOnPsArgs as created by the createMatchOnPsArgs() function.
cohortMethodData	An object of type CohortMethodData as generated using getDbCohortMethodData() . Needed when additionally matching on covariate IDs.

Details

The data frame should have the following three columns:

- `rowId` (numeric): A unique identifier for each row (e.g. the person ID).
- `treatment` (integer): Column indicating whether the person is in the target (1) or comparator (0) group.
- `propensityScore` (numeric): Propensity score.

The default caliper (0.2 on the standardized logit scale) is the one recommended by Austin (2011).

Value

Returns a data frame with the same columns as the input data plus one extra column: `stratumId`. Any rows that could not be matched are removed

References

Rassen JA, Shelat AA, Myers J, Glynn RJ, Rothman KJ, Schneeweiss S. (2012) One-to-many propensity score matching in cohort studies, *Pharmacoepidemiology and Drug Safety*, May, 21 Suppl 2:69-80.

Austin, PC. (2011) Optimal caliper widths for propensity-score matching when estimating differences in means and differences in proportions in observational studies, *Pharmaceutical statistics*, March, 10(2):150-161.

Examples

```
rowId <- 1:5
treatment <- c(1, 0, 1, 0, 1)
propensityScore <- c(0, 0.1, 0.3, 0.4, 1)
age_group <- c(1, 1, 1, 1, 1)
data <- data.frame(
  rowId = rowId,
  treatment = treatment,
  propensityScore = propensityScore,
  age_group = age_group
)
result <- matchOnPs(data, createMatchOnPsArgs(
  caliper = 0,
  maxRatio = 1,
  matchColumns = "age_group"
))
```

migrateDataModel

Migrate Data model

Description

Migrate data from current state to next state

It is strongly advised that you have a backup of all data (either sqlite files, a backup database (in the case you are using a postgres backend) or have kept the csv/zip files from your data generation.

Usage

```
migrateDataModel(connectionDetails, databaseSchema, tablePrefix = "")
```

Arguments

connectionDetails	DatabaseConnector connection details object
databaseSchema	String schema where database schema lives
tablePrefix	(Optional) Use if a table prefix is used before table names (e.g. "cd_")

```
plotCovariateBalanceOfTopVariables
```

Plot variables with largest imbalance

Description

Create a plot showing those variables having the largest imbalance before matching, and those variables having the largest imbalance after matching. Requires running `computeCovariateBalance` first.

Usage

```
plotCovariateBalanceOfTopVariables(
  balance,
  n = 20,
  maxNameWidth = 100,
  title = NULL,
  fileName = NULL,
  beforeLabel = "before matching",
  afterLabel = "after matching"
)
```

Arguments

balance	A data frame created by the <code>computeCovariateBalance</code> function.
n	(Maximum) count of covariates to plot.
maxNameWidth	Covariate names longer than this number of characters are truncated to create a nicer plot.
title	Optional: the main title for the plot.
fileName	Name of the file where the plot should be saved, for example 'plot.png'. See the function <code>ggsave</code> in the <code>ggplot2</code> package for supported file formats.
beforeLabel	Label for identifying data before matching / stratification / trimming.
afterLabel	Label for identifying data after matching / stratification / trimming.

Value

A `ggplot` object. Use the [ggplot2::ggsave](#) function to save to file in a different format.

plotCovariateBalanceScatterPlot

Create a scatterplot of the covariate balance

Description

Create a scatterplot of the covariate balance, showing all variables with balance before and after matching on the x and y axis respectively. Requires running `computeCovariateBalance` first.

Usage

```
plotCovariateBalanceScatterPlot(
  balance,
  absolute = TRUE,
  threshold = 0,
  title = "Standardized difference of mean",
  fileName = NULL,
  beforeLabel = "Before matching",
  afterLabel = "After matching",
  showCovariateCountLabel = FALSE,
  showMaxLabel = FALSE,
  showUnbalanced = FALSE
)
```

Arguments

<code>balance</code>	A data frame created by the <code>computeCovariateBalance</code> function.
<code>absolute</code>	Should the absolute value of the difference be used?
<code>threshold</code>	Show a threshold value for after matching standardized difference.
<code>title</code>	The main title for the plot.
<code>fileName</code>	Name of the file where the plot should be saved, for example 'plot.png'. See the function <code>ggsave</code> in the <code>ggplot2</code> package for supported file formats.
<code>beforeLabel</code>	Label for the x-axis.
<code>afterLabel</code>	Label for the y-axis.
<code>showCovariateCountLabel</code>	Show a label with the number of covariates included in the plot?
<code>showMaxLabel</code>	Show a label with the maximum absolute standardized difference after matching/stratification?
<code>showUnbalanced</code>	Show covariates that are considered unbalanced with a different color?

Value

A ggplot object. Use the [ggplot2::ggsave](#) function to save to file in a different format.

plotCovariatePrevalence

Plot covariate prevalence

Description

Plot prevalence of binary covariates in the target and comparator cohorts, before and after matching. Requires running `computeCovariateBalance` first.

Usage

```
plotCovariatePrevalence(
  balance,
  threshold = 0,
  title = "Covariate prevalence",
  fileName = NULL,
  beforeLabel = "Before matching",
  afterLabel = "After matching",
  targetLabel = "Target",
  comparatorLabel = "Comparator"
)
```

Arguments

<code>balance</code>	A data frame created by the <code>computeCovariateBalance</code> function.
<code>threshold</code>	A threshold value for standardized difference. When exceeding the threshold, covariates will be marked in a different color. If <code>threshold = 0</code> , no color coding will be used.
<code>title</code>	The main title for the plot.
<code>fileName</code>	Name of the file where the plot should be saved, for example 'plot.png'. See the function <code>ggsave</code> in the <code>ggplot2</code> package for supported file formats.
<code>beforeLabel</code>	Label for the before matching / stratification panel.
<code>afterLabel</code>	Label for the after matching / stratification panel.
<code>targetLabel</code>	Label for the x-axis.
<code>comparatorLabel</code>	Label for the y-axis.

Value

A ggplot object. Use the [ggplot2::ggsave](#) function to save to file in a different format.

`plotFollowUpDistribution`*Plot the distribution of follow-up time*

Description

Plot the distribution of follow-up time

Usage

```
plotFollowUpDistribution(  
  population,  
  targetLabel = "Target",  
  comparatorLabel = "Comparator",  
  yScale = "percent",  
  logYScale = FALSE,  
  dataCutoff = 0.95,  
  title = NULL,  
  fileName = NULL  
)
```

Arguments

population	A data frame describing the study population as created using the createStudyPopulation function. This should at least have these columns: treatment, timeAtRisk.
targetLabel	A label to us for the target cohort.
comparatorLabel	A label to us for the comparator cohort.
yScale	Should be either 'percent' or 'count'.
logYScale	Should the Y axis be on the log scale?
dataCutoff	Fraction of the data (number censored) after which the graph will not be shown.
title	The main title of the plot.
fileName	Name of the file where the plot should be saved, for example 'plot.png'. See the function <code>ggsave</code> in the <code>ggplot2</code> package for supported file formats.

Details

Plot the distribution of follow-up time, stratified by treatment group. Follow-up time is defined as time-at-risk, so not censored at the outcome.

Value

A ggplot object. Use the [ggsave](#) function to save to file in a different format.

plotKaplanMeier	<i>Plot the Kaplan-Meier curve</i>
-----------------	------------------------------------

Description

plotKaplanMeier creates the Kaplan-Meier (KM) survival plot. Based (partially) on recommendations in Pocock et al (2002).

When variable-sized strata are detected, an adjusted KM plot is computed to account for stratified data, as described in Galimberti et al (2002), using the closed form variance estimator described in Xie et al (2005).

Usage

```
plotKaplanMeier(
  population,
  censorMarks = FALSE,
  confidenceIntervals = TRUE,
  includeZero = FALSE,
  dataTable = TRUE,
  dataCutoff = 0.9,
  targetLabel = "Treated",
  comparatorLabel = "Comparator",
  title = NULL,
  fileName = NULL
)
```

Arguments

population	A population object generated by createStudyPopulation, potentially filtered by other functions.
censorMarks	Whether or not to include censor marks in the plot.
confidenceIntervals	Plot 95 percent confidence intervals? Default is TRUE, as recommended by Pocock et al.
includeZero	Should the y axis include zero, or only go down to the lowest observed survival? The default is FALSE, as recommended by Pocock et al.
dataTable	Should the numbers at risk be shown in a table? Default is TRUE, as recommended by Pocock et al.
dataCutoff	Fraction of the data (number censored) after which the graph will not be shown. The default is 90 percent as recommended by Pocock et al.
targetLabel	A label to us for the target cohort.
comparatorLabel	A label to us for the comparator cohort.
title	The main title of the plot.
fileName	Name of the file where the plot should be saved, for example 'plot.png'. See the function ggsave in the ggplot2 package for supported file formats.

Value

A ggplot object. Use the [ggsave](#) function to save to file in a different format.

References

Pocock SJ, Clayton TC, Altman DG. (2002) Survival plots of time-to-event outcomes in clinical trials: good practice and pitfalls, *Lancet*, 359:1686-89.

Galimberti S, Sasieni P, Valsecchi MG (2002) A weighted Kaplan-Meier estimator for matched data with application to the comparison of chemotherapy and bone-marrow transplant in leukaemia. *Statistics in Medicine*, 21(24):3847-64.

Xie J, Liu C. (2005) Adjusted Kaplan-Meier estimator and log-rank test with inverse probability of treatment weighting for survival data. *Statistics in Medicine*, 26(10):2276.

plotPs	<i>Plot the propensity score distribution</i>
--------	---

Description

Plots the propensity (or preference) score distribution.

Usage

```
plotPs(
  data,
  unfilteredData = NULL,
  scale = "preference",
  type = "density",
  binWidth = 0.05,
  targetLabel = "Target",
  comparatorLabel = "Comparator",
  showCountsLabel = FALSE,
  showAucLabel = FALSE,
  showEquipoiseLabel = FALSE,
  equipoiseBounds = c(0.3, 0.7),
  unitOfAnalysis = "subjects",
  title = NULL,
  fileName = NULL
)
```

Arguments

data	A data frame with at least the two columns described below
unfilteredData	To be used when computing preference scores on data from which subjects have already been removed, e.g. through trimming and/or matching. This data frame should have the same structure as data.
scale	The scale of the graph. Two scales are supported: scale = 'propensity' or scale = 'preference'. The preference score scale is defined by Walker et al (2013).

type	Type of plot. Four possible values: type = 'density' type = 'histogram', type = 'histogramCount', or type = 'histogramProportion'. 'histogram' defaults to 'histogramCount'.
binWidth	For histograms, the width of the bins
targetLabel	A label to us for the target cohort.
comparatorLabel	A label to us for the comparator cohort.
showCountsLabel	Show subject counts?
showAucLabel	Show the AUC?
showEquipoiseLabel	Show the percentage of the population in equipoise?
equipoiseBounds	The bounds on the preference score to determine whether a subject is in equipoise.
unitOfAnalysis	The unit of analysis in the input data. Defaults to 'subjects'.
title	Optional: the main title for the plot.
fileName	Name of the file where the plot should be saved, for example 'plot.png'. See the function <code>ggplot2::ggsave()</code> for supported file formats.

Details

The data frame should have a least the following two columns:

- treatment (integer): Column indicating whether the person is in the target (1) or comparator (0) group
- propensityScore (numeric): Propensity score

Value

A ggplot object. Use the `ggplot2::ggsave()` function to save to file in a different format.

References

Walker AM, Patrick AR, Lauer MS, Hornbrook MC, Marin MG, Platt R, Roger VL, Stang P, and Schneeweiss S. (2013) A tool for assessing the feasibility of comparative effectiveness research, Comparative Effective Research, 3, 11-20

Examples

```
treatment <- rep(0:1, each = 100)
propensityScore <- c(rnorm(100, mean = 0.4, sd = 0.25), rnorm(100, mean = 0.6, sd = 0.25))
data <- data.frame(treatment = treatment, propensityScore = propensityScore)
data <- data[data$propensityScore > 0 & data$propensityScore < 1, ]
plotPs(data)
```

plotTimeToEvent	<i>Plot time-to-event</i>
-----------------	---------------------------

Description

Plot time-to-event

Usage

```
plotTimeToEvent(
  cohortMethodData,
  population = NULL,
  outcomeId = NULL,
  minDaysAtRisk = 1,
  riskWindowStart = 0,
  startAnchor = "cohort start",
  riskWindowEnd = 0,
  endAnchor = "cohort end",
  censorAtNewRiskWindow = FALSE,
  periodLength = 7,
  numberOfPeriods = 52,
  highlightExposedEvents = TRUE,
  includePostIndexTime = TRUE,
  showFittedLines = TRUE,
  targetLabel = "Target",
  comparatorLabel = "Comparator",
  title = NULL,
  fileName = NULL
)
```

Arguments

cohortMethodData	An object of type CohortMethodData as generated using getDbCohortMethodData() .
population	If specified, this population will be used as the starting point instead of the cohorts in the cohortMethodData object.
outcomeId	The ID of the outcome. If NULL, no outcome-specific transformations will be performed.
minDaysAtRisk	The minimum required number of days at risk.
riskWindowStart	The start of the risk window (in days) relative to the startAnchor.
startAnchor	The anchor point for the start of the risk window. Can be "cohort start" or "cohort end".
riskWindowEnd	The end of the risk window (in days) relative to the endAnchor.
endAnchor	The anchor point for the end of the risk window. Can be "cohort start" or "cohort end".
censorAtNewRiskWindow	If a subject is in multiple cohorts, should time-at-risk be censored when the new time-at-risk starts to prevent overlap?

periodLength	The length in days of each period shown in the plot.
numberOfPeriods	Number of periods to show in the plot. The periods are equally divided before and after the index date.
highlightExposedEvents	(logical) Highlight event counts during exposure in a different color?
includePostIndexTime	(logical) Show time after the index date?
showFittedLines	(logical) Fit lines to the proportions and show them in the plot?
targetLabel	A label to us for the target cohort.
comparatorLabel	A label to us for the comparator cohort.
title	Optional: the main title for the plot.
fileName	Name of the file where the plot should be saved, for example 'plot.png'. See ggplot2::ggsave() for supported file formats.

Details

Creates a plot showing the number of events over time in the target and comparator cohorts, both before and after index date. The plot also distinguishes between events inside and outside the time-at-risk period. This requires the user to (re)specify the time-at-risk using the same arguments as the [createStudyPopulation\(\)](#) function. Note that it is not possible to specify that people with the outcome prior should be removed, since the plot will show these prior events.

Value

A ggplot object. Use the [ggplot2::ggsave\(\)](#) function to save to file in a different format.

runCmAnalyses	<i>Run a list of analyses</i>
---------------	-------------------------------

Description

Run a list of analyses

Usage

```
runCmAnalyses(
  connectionDetails,
  cdmDatabaseSchema,
  tempEmulationSchema = getOption("sqlRenderTempEmulationSchema"),
  exposureDatabaseSchema = cdmDatabaseSchema,
  exposureTable = "drug_era",
  outcomeDatabaseSchema = cdmDatabaseSchema,
  outcomeTable = "condition_occurrence",
  nestingCohortDatabaseSchema = cdmDatabaseSchema,
  nestingCohortTable = "cohort",
  outputFolder = "./CohortMethodOutput",
  multiThreadingSettings = createMultiThreadingSettings(),
  cmAnalysesSpecifications
)
```

Arguments

connectionDetails

An R object of type connectionDetails created using the [DatabaseConnector::createConnectionDetails\(\)](#) function.

cdmDatabaseSchema

The name of the database schema that contains the OMOP CDM instance. Requires read permissions to this database. On SQL Server, this should specify both the database and the schema, so for example 'cdm_instance.dbo'.

tempEmulationSchema

Some database platforms like Oracle and Impala do not truly support temp tables. To emulate temp tables, provide a schema with write privileges where temp tables can be created.

exposureDatabaseSchema

The name of the database schema that is the location where the exposure data used to define the exposure cohorts is available. If exposureTable = DRUG_ERA, exposureDatabaseSchema is not used by assumed to be cdmSchema. Requires read permissions to this database.

exposureTable

The tablename that contains the exposure cohorts. If exposureTable <> DRUG_ERA, then expectation is exposureTable has format of COHORT table: COHORT_DEFINITION_ID, SUBJECT_ID, COHORT_START_DATE, COHORT_END_DATE.

outcomeDatabaseSchema

The name of the database schema that is the location where the data used to define the outcome cohorts is available. If exposureTable = CONDITION_ERA, exposureDatabaseSchema is not used by assumed to be cdmSchema. Requires read permissions to this database.

outcomeTable

The tablename that contains the outcome cohorts. If outcomeTable <> CONDITION_OCCURRENCE, then expectation is outcomeTable has format of COHORT table: COHORT_DEFINITION_ID, SUBJECT_ID, COHORT_START_DATE, COHORT_END_DATE.

nestingCohortDatabaseSchema

The name of the database schema that is the location where the data used to define the nesting cohorts is available.

nestingCohortTable

The tablename that contains the nesting cohorts. Must have the format of COHORT table: COHORT_DEFINITION_ID, SUBJECT_ID, COHORT_START_DATE, COHORT_END_DATE.

outputFolder

Name of the folder where all the outputs will written to.

multiThreadingSettings

An object of type CmMultiThreadingSettings as created using the [createMultiThreadingSettings\(\)](#) or [createDefaultMultiThreadingSettings\(\)](#) functions.

cmAnalysesSpecifications

An object of type CmAnalysesSpecifications as created using the [createCmAnalysesSpecifications\(\)](#) function.

Details

Run a list of analyses for the target-comparator-outcomes of interest. This function will run all specified analyses against all hypotheses of interest, meaning that the total number of outcome models is $\text{length}(\text{cmAnalysisList}) * \text{length}(\text{targetComparatorOutcomesList})$ (if all analyses specify an outcome model should be fitted). When you provide several analyses it will determine whether any of the analyses have anything in common, and will take advantage of this fact. For example, if

we specify several analyses that only differ in the way the outcome model is fitted, then this function will extract the data and fit the propensity model only once, and re-use this in all the analysis.

After completion, a tibble containing references to all generated files can be obtained using the [getFileReference\(\)](#) function. A summary of the analysis results can be obtained using the [getResultsSummary\(\)](#) function. Diagnostics can be loaded using the [getDiagnosticsSummary\(\)](#) function.

Value

A tibble describing for each target-comparator-outcome-analysisId combination where the intermediary and outcome model files can be found, relative to the outputFolder.

saveCmAnalysisList	<i>Save a list of CmAnalysis to file</i>
--------------------	--

Description

Write a list of objects of type CmAnalysis to file. The file is in JSON format.

Usage

```
saveCmAnalysisList(CmAnalysisList, file)
```

Arguments

CmAnalysisList	A list of objects of type CmAnalysis as created using the createCmAnalysis() function.
file	The name of the file where the results will be written

saveCohortMethodData	<i>Save the cohort method data to file</i>
----------------------	--

Description

Saves an object of type [CohortMethodData](#) to a file.

Usage

```
saveCohortMethodData(cohortMethodData, file)
```

Arguments

cohortMethodData	An object of type CohortMethodData as generated using getDbCohortMethodData() .
file	The name of the file where the data will be written. If the file already exists it will be overwritten.

Value

Returns no output.

`saveTargetComparatorOutcomesList`*Save a list of TargetComparatorOutcomes to file*

Description

Write a list of objects of type TargetComparatorOutcomes to file. The file is in JSON format.

Usage

```
saveTargetComparatorOutcomesList(targetComparatorOutcomesList, file)
```

Arguments

targetComparatorOutcomesList

A list of objects of type TargetComparatorOutcomes as created using the [createTargetComparatorOutcomesList\(\)](#) function.

file

The name of the file where the results will be written

`simulateCohortMethodData`*Generate simulated data*

Description

Creates a [CohortMethodData](#) object with simulated data.

Usage

```
simulateCohortMethodData(profile, n = 10000)
```

Arguments

profile

An object of type CohortMethodDataSimulationProfile as generated using the [createCohortMethodDataSimulationProfile\(\)](#) function.

n

The size of the population to be generated.

Details

This function generates simulated data that is in many ways similar to the original data on which the simulation profile is based. The contains same outcome, comparator, and outcome concept IDs, and the covariates and their 1st order statistics should be comparable.

Value

An object of type [CohortMethodData](#).

stratifyByPs	<i>Stratify persons by propensity score</i>
--------------	---

Description

Use the provided propensity scores to stratify persons. Additional stratification variables for stratifications can also be used.

Usage

```
stratifyByPs(
  population,
  stratifyByPsArgs = createStratifyByPsArgs(),
  cohortMethodData = NULL
)
```

Arguments

population	A data frame with the three columns described below
stratifyByPsArgs	An object of type <code>StratifyByPsArgs</code> as created by the <code>createStratifyByPsArgs()</code> function.
cohortMethodData	An object of type <code>CohortMethodData</code> as generated using <code>getDbCohortMethodData()</code> . Needed when additionally matching on covariate IDs.

Details

The data frame should have the following three columns:

- `rowId` (numeric): A unique identifier for each row (e.g. the person ID).
- `treatment` (integer): Column indicating whether the person is in the target (1) or comparator (0) group.
- `propensityScore` (numeric): Propensity score.

Value

Returns a tibble with the same columns as the input data plus one extra column: `stratumId`.

Examples

```
rowId <- 1:200
treatment <- rep(0:1, each = 100)
propensityScore <- c(runif(100, min = 0, max = 1), runif(100, min = 0, max = 1))
data <- data.frame(rowId = rowId, treatment = treatment, propensityScore = propensityScore)
result <- stratifyByPs(data, createStratifyByPsArgs(numberOfStrata = 5))
```

trimByPs	<i>Trim persons by propensity score</i>
----------	---

Description

Use the provided propensity scores to trim subjects with extreme scores or weights.

Usage

```
trimByPs(population, trimByPsArgs = createTrimByPsArgs(trimFraction = 0.05))
```

Arguments

population	A data frame with the three columns described below
trimByPsArgs	An object of type trimByPsArgs as created by the createTrimByPsArgs() function.

Details

The data frame should have the following three columns:

- `rowId` (numeric): A unique identifier for each row (e.g. the person ID).
- `treatment` (integer): Column indicating whether the person is in the target (1) or comparator (0) group.
- `propensityScore` (numeric): Propensity score.

Value

Returns a tibble with the same three columns as the input.

Examples

```
rowId <- 1:2000
treatment <- rep(0:1, each = 1000)
propensityScore <- c(runif(1000, min = 0, max = 1), runif(1000, min = 0, max = 1))
iptw <- ifelse(treatment == 1,
               mean(treatment == 1) / propensityScore,
               mean(treatment == 0) / (1 - propensityScore))
data <- data.frame(rowId = rowId,
                  treatment = treatment,
                  propensityScore = propensityScore,
                  iptw = iptw)
result1 <- trimByPs(data, createTrimByPsArgs(trimFraction = 0.05))
result2 <- trimByPs(data, createTrimByPsArgs(equipoiseBounds = c(0.3, 0.7)))
result3 <- trimByPs(data, createTrimByPsArgs(maxWeight = 10))
```

truncateIptw	<i>Truncate IPTW values</i>
--------------	-----------------------------

Description

Set the inverse probability of treatment weights (IPTW) to the user-specified threshold if it exceeds said threshold.

Usage

```
truncateIptw(population, truncateIptwArgs = createTruncateIptwArgs())
```

Arguments

population	A data frame with at least the two columns described in the details.
truncateIptwArgs	An object of type TruncateIptwArgs as created by the createTruncateIptwArgs() function.

Details

The data frame should have the following two columns:

- treatment (integer): Column indicating whether the person is in the target (1) or comparator (0) group.
- iptw (numeric): Propensity score.

Value

Returns a tibble with the same columns as the input.

Examples

```
rowId <- 1:2000
treatment <- rep(0:1, each = 1000)
iptw <- 1 / c(runif(1000, min = 0, max = 1), runif(1000, min = 0, max = 1))
data <- data.frame(rowId = rowId, treatment = treatment, iptw = iptw)
result <- truncateIptw(data)
```

uploadResults	<i>Upload results to the database server.</i>
---------------	---

Description

Requires the results data model tables have been created using the [createResultsDataModel](#) function.

Usage

```

uploadResults(
  connectionDetails,
  schema,
  zipFileName,
  forceOverWriteOfSpecifications = FALSE,
  purgeSiteDataBeforeUploading = TRUE,
  tempFolder = tempdir(),
  tablePrefix = "",
  ...
)

```

Arguments

connectionDetails	An object of type connectionDetails as created using the createConnectionDetails function in the DatabaseConnector package.
schema	The schema on the server where the tables have been created.
zipFileName	The name of the zip file.
forceOverWriteOfSpecifications	If TRUE, specifications of the phenotypes, cohort definitions, and analysis will be overwritten if they already exist on the database. Only use this if these specifications have changed since the last upload.
purgeSiteDataBeforeUploading	If TRUE, before inserting data for a specific databaseId all the data for that site will be dropped. This assumes the input zip file contains the full data for that data site.
tempFolder	A folder on the local file system where the zip files are extracted to. Will be cleaned up when the function is finished. Can be used to specify a temp folder on a drive that has sufficient space if the default system temp space is too limited.
tablePrefix	(Optional) string to insert before table names for database table names
...	See ResultModelManager::uploadResults

Index

- * **datasets**
 - cohortMethodDataSimulationProfile, 5
- adjustedKm, 4
- Andromeda, 5
- checkCmInstallation, 4
- CohortMethodData, 6, 15, 26, 29, 34, 37, 40, 41, 43, 44, 53, 56–58
- CohortMethodData
 - (CohortMethodData-class), 5
- CohortMethodData-class, 5
- cohortMethodDataSimulationProfile, 5
- computeCovariateBalance, 5
- computeCovariateBalance(), 13, 16, 39
- computeEquipoise, 8
- computeMdr, 8
- computePsAuc, 9
- convertUntypedListToCmAnalysesSpecifications, 10
- CovariateData, 5
- createCmAnalysesSpecifications, 11
- createCmAnalysis, 11, 12
- createCmAnalysis(), 56
- createCmDiagnosticThresholds, 13
- createCmDiagnosticThresholds(), 11
- createCmTable1, 14, 37
- createCohortMethodDataSimulationProfile, 15
- createCohortMethodDataSimulationProfile(), 57
- createComputeCovariateBalanceArgs, 16
- createConnectionDetails, 61
- createCreatePsArgs, 17
- createCreatePsArgs(), 26
- createCreateStudyPopulationArgs, 18
- createCreateStudyPopulationArgs(), 26, 29
- createDefaultMultiThreadingSettings, 19
- createDefaultMultiThreadingSettings(), 25, 55
- createFitOutcomeModelArgs, 20
- createFitOutcomeModelArgs(), 34
- createGetDbCohortMethodDataArgs, 21
- createGetDbCohortMethodDataArgs(), 30, 36
- createMatchOnPsArgs, 23
- createMatchOnPsArgs(), 44
- createMultiThreadingSettings, 24
- createMultiThreadingSettings(), 19, 55
- createOutcome, 25
- createOutcome(), 30
- createPs, 26
- createPs(), 12, 17, 41
- createResultsDataModel, 27, 60
- createStratifyByPsArgs, 28
- createStudyPopulation, 9, 28, 38, 49
- createStudyPopulation(), 12, 18, 34, 54
- createTargetComparatorOutcomes, 11, 29
- createTargetComparatorOutcomes(), 26, 57
- createTrimByPsArgs, 30
- createTrimByPsArgs(), 59
- createTruncateIptwArgs, 31
- createTruncateIptwArgs(), 60
- Cyclops::createControl(), 21
- Cyclops::createPrior(), 21
- DatabaseConnector::createConnectionDetails, 27
- DatabaseConnector::createConnectionDetails(), 36, 55
- drawAttritionDiagram, 32
- exportToCsv, 33
- exportToCsv(), 13
- FeatureExtraction::getDefaultTable1Specifications(), 16
- fitOutcomeModel, 33
- fitOutcomeModel(), 13, 20, 40
- getAttritionTable, 34
- getDataMigrator, 35
- getDbCohortMethodData, 35
- getDbCohortMethodData(), 5, 6, 12, 15, 21, 26, 29, 34, 40, 41, 44, 53, 56, 58

[getDefaultCmTable1Specifications](#), 37
[getDiagnosticsSummary](#), 37
[getDiagnosticsSummary\(\)](#), 56
[getFileReference](#), 38
[getFileReference\(\)](#), 56
[getFollowUpDistribution](#), 38
[getGeneralizabilityTable](#), 39
[getInteractionResultsSummary](#), 40
[getOutcomeModel](#), 40
[getPsModel](#), 41
[getResultsDataModelSpecifications](#), 41
[getResultsSummary](#), 42
[getResultsSummary\(\)](#), 56
[ggplot2::ggsave](#), 46–48
[ggplot2::ggsave\(\)](#), 52, 54
[ggsave](#), 32, 49, 51

[isCohortMethodData](#), 42

[loadCmAnalysisList](#), 43
[loadCohortMethodData](#), 43
[loadCohortMethodData\(\)](#), 5
[loadTargetComparatorOutcomesList](#), 44

[matchOnPs](#), 44
[matchOnPs\(\)](#), 13, 23
[migrateDataModel](#), 45

[plotCovariateBalanceOfTopVariables](#), 46
[plotCovariateBalanceScatterPlot](#), 47
[plotCovariatePrevalence](#), 48
[plotFollowUpDistribution](#), 49
[plotKaplanMeier](#), 50
[plotPs](#), 51
[plotTimeToEvent](#), 53

[runCmAnalyses](#), 13, 54
[runCmAnalyses\(\)](#), 13, 30, 33

[saveCmAnalysisList](#), 56
[saveCohortMethodData](#), 56
[saveCohortMethodData\(\)](#), 5
[saveTargetComparatorOutcomesList](#), 57
[show, CohortMethodData-method](#)
 (CohortMethodData-class), 5
[simulateCohortMethodData](#), 57
[simulateCohortMethodData\(\)](#), 16
[stratifyByPs](#), 58
[stratifyByPs\(\)](#), 13, 28
[summary, CohortMethodData-method](#)
 (CohortMethodData-class), 5

[trimByPs](#), 59
[trimByPs\(\)](#), 12, 30

[truncateIptw](#), 60
[truncateIptw\(\)](#), 13, 31

[uploadResults](#), 60